

FIXED-PARAMETER TRACTABLE COMPUTATION OF RESHETIKHIN–TURAEV KNOT POLYNOMIALS VIA TENSOR NETWORKS

SHANA YUNSHENG LI

ABSTRACT. We give a thorough analysis of the time complexity of computing Reshetikhin–Turaev knot polynomials via tensor contractions on the associated tensor networks, showing that the computation is fixed-parameter tractable with respect to a parameter at most linear in the tree-width of the input knot diagram. When combined with existing approximation algorithms for tree decomposition, this recovers the sub-exponential bound $e^{O(\sqrt{n})}$ for the time complexity of computing any Reshetikhin–Turaev knot polynomial. We accompany this paper with an implementation of such an algorithm in `SnapPy`, which computes any Reshetikhin–Turaev knot polynomial given its R -matrix and ribbon element.

CONTENTS

1. Introduction	1
2. Setup	5
2.1. Long knot diagrams and rotation numbers	5
2.2. The Reshetikhin–Turaev functor	6
2.3. Tensor networks and contractions	7
3. Bounding the time complexity	9
3.1. Contraction-width	9
3.2. Proof of Theorem 1.2	10
3.3. Proof of Corollary 1.4	11
Acknowledgements	12
References	12

1. INTRODUCTION

The discovery of the Jones polynomial [Jon87] revealed deep connections of low dimensional topology with physics [Wit89], leading to the birth of quantum topology. Building upon this foundation, the Reshetikhin–Turaev functor [Tur88a, RT90] was later established as a powerful and systematic approach to defining numerous knot polynomials, including the colored Jones polynomials [KR89, Tur88b] and the colored HOMFLY polynomials. As is the case for many invariants, efficient computation is essential for studying their properties [BN05, Shu11, GL26, GLY], and is thus naturally an important topic. However, it was

Date: June 28, 2026.

Key words and phrases: knots, knot polynomials, Reshetikhin–Turaev functor, fixed-parameter tractable, algorithm, computational topology, quantum topology .

later understood that Reshetikhin–Turaev knot polynomials are generally $\#P$ -hard to compute, as it was proved for the Jones polynomial [JVV90], one of the simplest Reshetikhin–Turaev knot polynomials. $\#P$ -hard problems are analogues of NP-hard problems, where one computes a value (in this case, the knot polynomial) rather than deciding a yes/no question. Thus for $\#P$ -hard problems, the best algorithm one could expect is *fixed-parameter tractable* [CFK⁺15], which in general means that the input can be subdivided into families $\mathcal{F}_i$ parametrized by $i \in \mathbb{N}$, such that the time complexity for an input in $\mathcal{F}_i$ of size n is

$$O(f(i) \cdot p(n)), \quad (1)$$

where p is a polynomial and f is an arbitrary computable function. We say that an algorithm satisfying (1) is *fixed-parameter tractable with respect to i* .

For computations of knot invariants, i is often a graph-theoretic width of planar diagrams of knots. Some relevant examples of widths are the *tree-width*, *cut-width*, *carving-width*, and, proposed in this paper, *contraction-width*. Among them, tree-width is the most extensively studied; we briefly recall the definition of tree-width below and list its relationship with other widths in Table 1. The contraction-width plays a central role in this paper, and its definition is given in Section 3.1.

Definition 1.1. A tree decomposition of a graph $\Gamma = (V, E)$ is a tree T whose vertices consist of subsets X_i of V such that

- (a) $\bigcup_i X_i = V$.
- (b) If $v \in X_i \cap X_j$, then any X_k in the path in T connecting X_i and X_j also contains v .
- (c) For every edge $\{v, w\} \in E$, there exists X_i such that $v, w \in X_i$.

The width of a tree decomposition is $\max_i |X_i| - 1$. The *tree-width* of Γ is the minimum width among all possible tree decompositions of Γ .

Name of width	Notation	Relationship with tree-width	Asymptotic behaviour	Reference
Tree-width	$tw(\Gamma)$	itself	$O(\sqrt{n})$	[LT79]
Cut-width	$c(\Gamma)$	$c(\Gamma) = O(tw(\Gamma) \cdot \Delta(\Gamma) \cdot \log n)$	$O(\sqrt{n} \log n)$	[KS93]
Carving-width	$cw(\Gamma)$	$\frac{2}{3}(tw(\Gamma) + 1) \leq cw(\Gamma) \leq \Delta(\Gamma) \cdot (tw(\Gamma) + 1)$	$O(\sqrt{n})$	[Bie90]
Contraction-width	$w(\Gamma)$	$w(\Gamma) \leq \Delta(\Gamma) \cdot (tw(\Gamma) + 1)$	$O(\sqrt{n})$	Lemma 1.3

TABLE 1. Different widths and their relationship with the tree-width, where Γ is a planar graph with n vertices and maximum degree $\Delta(\Gamma)$.

For the fixed-parameter tractable algorithms discussed in this paper, we have $f(i) = e^{O(i)}$ and $i = O(\sqrt{n})$ in terms of (1). Previously, fixed-parameter tractable algorithms were first established for the Jones polynomial [?] and then for all Reshetikhin–Turaev knot polynomials [Mar21], the latter being fixed-parameter tractable with respect to the carving-width. It follows that the algorithm in [Mar21] gives a sub-exponential bound $e^{O(\sqrt{n})}$ for computing Reshetikhin–Turaev knot polynomials. However, the algorithm in [Mar21] involves classifying morphism compositions into 7 elementary pieces and dealing with them case by case. To

the best of the author's knowledge, no publicly available implementation of the algorithm in [Mar21] exists as of this writing.

On the other hand, the computation of Reshetikhin–Turaev knot polynomials naturally translates into tensor contractions on the corresponding tensor networks. Tensor networks have been intensively studied in recent years [MS08, Orú19], and the implementation of tensor contractions is straightforward. Experiments have revealed such computations to be quite efficient for computing knot polynomials [MK19, GL26]. The author gave an implementation in [GL24] accompanying [GL26], using which the V_2 -polynomial (also known as the 2-colored Links–Gould polynomial) was computed for millions of knots including all those up to 16 crossings; the computed data confirmed, for all 352 million knots up to 19 crossings, that the V_2 -polynomial detects the genus of knots. However, no thorough analysis of the time complexity of the tensor network approach has appeared in the literature.

In this note, we show that the computation of Reshetikhin–Turaev knot polynomials via tensor networks is in fact fixed-parameter tractable with respect to the contraction-width, and recovers the sub-exponential bound given in [Mar21]. The contraction-width, defined in Section 3.1, was formalized by the author based on the practice in [GL26], and naturally leads to the results here. More precisely, let RT be a k -variable Reshetikhin–Turaev knot polynomials defined by an R -matrix $R \in \text{Aut}(V \otimes V)$ and a ribbon element $h \in \text{Aut}(V)$, and let d be the dimension of V , we show the following:

Theorem 1.2. *For any oriented knot diagram D with n crossings and a contraction sequence $\mathcal{S}$ on D with contraction-width w , computing $RT(D)$ by contracting along $\mathcal{S}$ is*

$$O(d^w n^{k+2} (\log n)^3). \quad (2)$$

With minimal tweaks, the code in [GL24] turns into an implementation of Theorem 1.2. The author has also implemented an improved version of [GL24] in `SnapPy` [CDGW]¹, with which we benchmarked the running time on various sets of knots and produced Figure 1.

The following is a straightforward consequence of [MS08], which we explain in Section 3.1:

Lemma 1.3. For any graph Γ ,

$$w(\Gamma) \leq \Delta(\Gamma) \cdot (tw(\Gamma) + 1), \quad (3)$$

where $\Delta(\Gamma)$ is the maximum degree of Γ .

Applying an algorithm which generates a contraction sequence which is approximately optimal, we can drop the requirement of the contraction sequence from Theorem 1.2, obtaining:

Corollary 1.4. For any oriented knot diagram D with n crossings, computing $RT(D)$ is

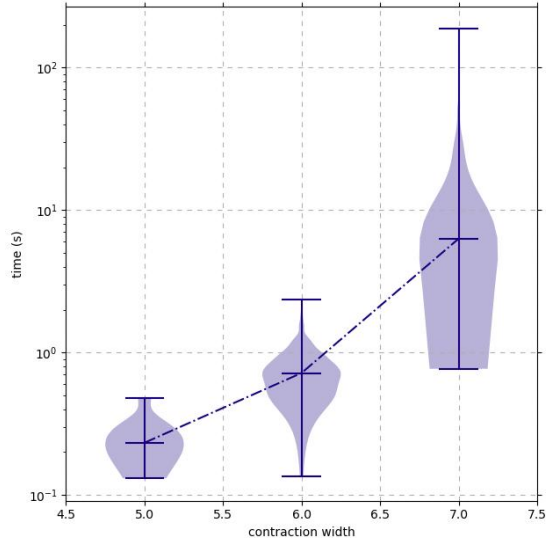
$$O(d^{2w(D)} n^{k+2} (\log n)^3 + 2^{10.8w(D)} n), \quad (4)$$

where $2^{10.8w(D)} n$ is the time complexity for finding the approximate contraction sequence.

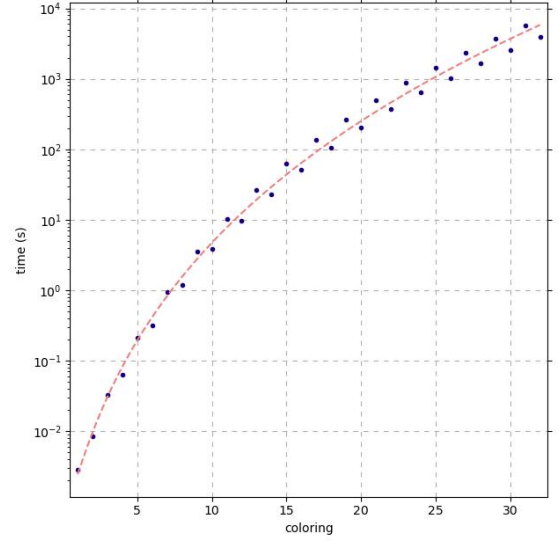
We remark that different versions of Corollary 1.4 can be obtained by using different approximation algorithms for contraction sequences, which we explain in Section 3.3.

Applying Lemma 1.3 and substituting $tw(D)$ with $O(\sqrt{n})$, Corollary 1.4 translates into

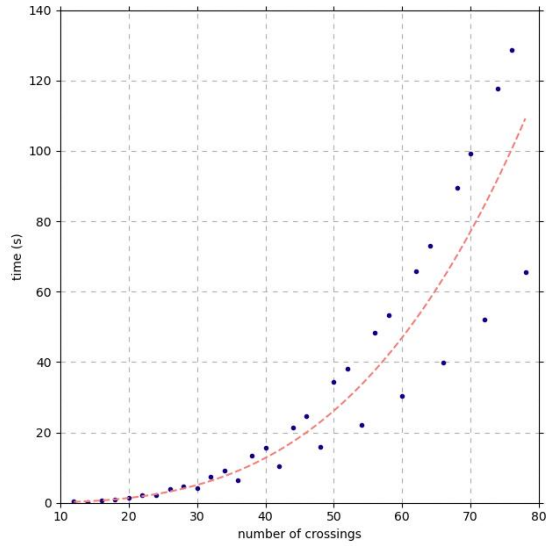
¹The author acknowledges the usage of Claude Code Opus 4.8 in assist of part of the implementation; the under-development fork is available at [Li26], which will be merged into `SnapPy` in the near future.



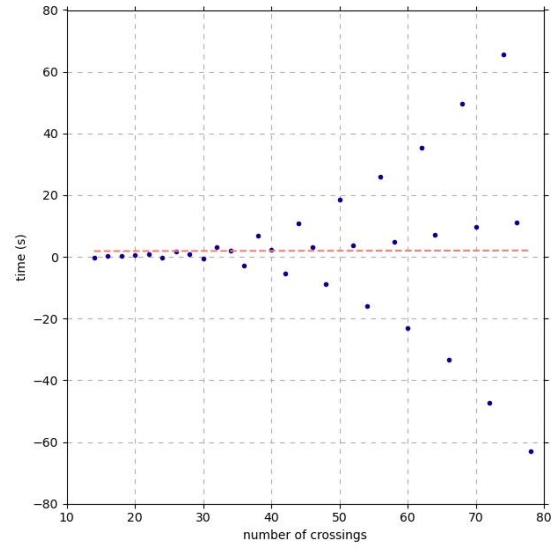
(A) The V_3 -polynomial for the first 1000 knots with 13 crossings in the knot table [HTW98], plotted in log scale.



(B) The n -colored Jones polynomial for the 11n34 knot as n ranges from 1 to 32, using a contraction sequence of width 7, plotted in log scale.



(C) The V_2 -polynomial for $(3, i)$ -torus links with $i = 6, \dots, 39$, all using contraction sequences of width 7.



(D) Consecutive differences of the computing times in Figure 1c.

FIGURE 1. Running time for computing polynomials on various sets of knots. For the V_n -polynomial, $k = 2$ and $d = 4n$; for the n -colored Jones polynomial, $k = 1$ and $d = n + 1$.

Corollary 1.5. For any oriented knot diagram D with n crossings, $RT(D)$ can be computed in time $e^{O(\sqrt{n})}$.

This recovers the sub-exponential bound given in [Mar21]. Note, however, that Theorem 1.2 is not necessarily equivalent to the results in [Mar21]. The author is not aware of a direct relationship between the carving-width and the contraction-width, even though they are both bounded by $\Delta(\Gamma) \cdot (tw(\Gamma) + 1)$ from above.

We give a brief review of the Reshetikhin–Turaev functor in Section 2, explaining how the computation translates into tensor contractions, and then give the detailed analysis for the time complexity in Section 3, proving Theorem 1.2. More details on the materials in Section 2 can be found in [GL26]. Throughout this paper, Reshetikhin–Turaev knot polynomials are assumed to be given by R -matrices and ribbon elements with matrix representations whose entries are Laurent polynomials with integer coefficients.

We remark that our results easily generalize to links and possibly with differently colored components, but we talk about knots exclusively for simplicity and because “knot polynomial” is a much more common phrase than “link polynomial”.

2. SETUP

We give some basic descriptions about our settings in this section, laying the groundwork for the discussion of time complexity in Section 3.

2.1. Long knot diagrams and rotation numbers. Given a planar diagram of an oriented knot, we convert it into an oriented long knot diagram by cutting an arc open and pulling the two open ends towards infinity in opposite directions without creating any self-intersection. Following the convention in [BNvdV24], we regard the out-pointing strand to be pulled vertically upward, and the other one vertically downward. We further require that the arcs always point upward in sufficiently small neighborhoods of each crossing by applying an ambient isotopy which may rotate the crossings.

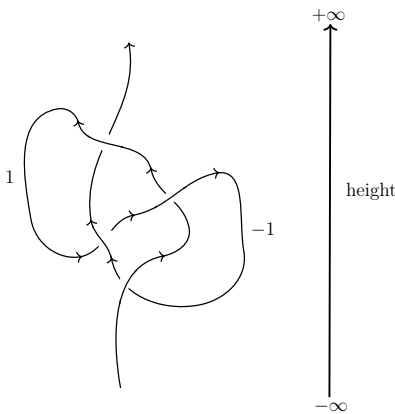


FIGURE 2. An oriented long knot diagram of the 4_1 knot, with nonzero rotation numbers labeled.

The vertical axis now gives us a natural height function, which restricts to a Morse function along the long knot diagram, whose local maxima and minima occur only on the arcs and never at the crossings. The rotation number associated to each arc in the oriented long knot diagram is defined as follows.

Definition 2.1. The *rotation number* of an arc $\mathcal{A}$ in an oriented long knot diagram equipped with a height function ℓ as described above, is the integer

$$\sum_{p \in \{d\ell=0\} \cap \mathcal{A}} (-1)^{\delta_\ell(p)} \varepsilon_{\mathcal{A}}(p),$$

where $\delta_\ell(p) = 1$ if p is a local maximum of ℓ and 0 otherwise; $\varepsilon_{\mathcal{A}}(p) = 1$ if the oriented arc $\mathcal{A}$ points to the right at p , and 0 otherwise.

Note that the rotation number is determined not only by the combinatorial information of the oriented long knot diagram, but also by how the diagram is embedded in the plane, inducing the height function. Given a planar diagram of an oriented knot with n crossings, there is an $O(n)$ algorithm that converts it into an oriented long knot diagram with rotation numbers in $\{0, \pm 1\}$ [BNvdV24]. A description of the algorithm can be found in [GL26, Section 2.2.1].

2.2. The Reshetikhin–Turaev functor. Depending on the context, the precise constructions of the Reshetikhin–Turaev functor vary, but they all share some common structures, of which only those relevant to computation are described here; we refer the reader to [Oht02, GK26] for in-depth introductions to the theoretical aspects.

The central idea of the Reshetikhin–Turaev functor is to view oriented long knot diagrams as an analogue of 1-dimensional cobordisms, assembled from elementary pieces, and to map the pieces into the category of vector spaces. The composition of the obtained linear maps is thus the desired knot invariant.

Therefore, as with any topological quantum field theory, the Reshetikhin–Turaev functor is determined by its image on those elementary pieces. The information needed consists of:

- A vector space V over a field $\mathbb{F}$.
- An R -matrix $R \in \text{Aut}(V \otimes V)$.
- A ribbon element $h \in \text{Aut}(V)$.

The Reshetikhin–Turaev functor is then determined as follows:

$$\begin{array}{cccccc}
 \uparrow \mapsto \begin{array}{c} V \\ \uparrow 1_V \\ V \end{array} & \curvearrowright \mapsto \begin{array}{c} V \otimes V \\ \uparrow R \\ V \otimes V \end{array} & \cup \mapsto \begin{array}{c} V^* \otimes V \\ \uparrow u \\ \mathbb{F} \end{array} & \cap \mapsto \begin{array}{c} V \otimes V^* \\ \uparrow u' \\ \mathbb{F} \end{array} \\
 \downarrow \mapsto \begin{array}{c} V^* \\ \uparrow 1_{V^*} \\ V^* \end{array} & \curvearrowright \mapsto \begin{array}{c} V \otimes V \\ \uparrow R^{-1} \\ V \otimes V \end{array} & \cup \mapsto \begin{array}{c} \mathbb{F} \\ \uparrow n \\ V \otimes V^* \end{array} & \cap \mapsto \begin{array}{c} \mathbb{F} \\ \uparrow n' \\ V^* \otimes V \end{array}
 \end{array}$$

where u and u' are defined by

$$u(1) = \sum_i e_i^* \otimes h(e_i), \quad u'(1) = \sum_i e_i \otimes e_i^*,$$

where $\{e_i\}$ is any basis of V and $\{e_i^*\}$ is the dual basis of $\{e_i\}$, and n and n' are defined by

$$n(x \otimes f) = f(h^{-1}(x)), \quad n'(f \otimes x) = f(x)$$

for any $x \in V$ and $f \in V^*$.

Since $(n \otimes 1_V) \circ (1_V \otimes u) = 1_V$ and $(n' \otimes 1_{V^*}) \circ (1_{V^*} \otimes u') = 1_{V^*}$, it is easy to see that the result of applying the Reshetikhin–Turaev functor is determined by the combinatorial information of the planar diagram along with the rotation numbers. With the rotation numbers living in $\{0, \pm 1\}$, the result can be described by $O(n)$ elementary pieces. Hence it is $O(n)$ to convert an oriented long knot diagram into compositions of linear maps as above.

The result of applying the Reshetikhin–Turaev functor to an oriented long knot diagram is a diagonalizable linear map in $\text{End}(V)$ whose eigenvalues are all the same. To obtain a knot polynomial, indeterminates are plugged into the definitions of R and h , turning the resulting linear map into a parametrized family, and the eigenvalue into the polynomial.

2.3. Tensor networks and contractions. We briefly review some standard terminology of tensor networks and tensor contractions. We refer the reader to [GL26] for slightly more detailed descriptions, and to [BC17] for an extensive and graphical introduction to this topic.

Definition 2.2. Let $\mathcal{R}$ be a (commutative) ring. An n -tensor T over $\mathcal{R}$ is a tuple $(T_{i_1, \dots, i_n})_{(i_1, \dots, i_n) \in \mathcal{S}}$ where $T_{i_1, \dots, i_n} \in \mathcal{R}$ and the index set $\mathcal{S}$ is of the form

$$\mathcal{S} = \{1, \dots, m_1\} \times \dots \times \{1, \dots, m_n\} \subset \mathbb{N}^n,$$

We call the integer m_k ($k \in \{1, \dots, n\}$) the dimension of the leg k of the tensor T .

Definition 2.3. A *tensor network* over a ring $\mathcal{R}$ is a (multi)graph Γ where each vertex v is associated with a tensor T_v over $\mathcal{R}$ whose legs are bijectively associated with the ends of the edges incident to v . Such an association must satisfy that for each edge in Γ , its two ends are associated with two different legs with the same dimension of one or two tensors.

Given a tensor network Γ over $\mathcal{R}$, for an n -tensor T and an n' -tensor T' in Γ , if two legs k of T and k' of T' are associated with the two ends of an edge e in Γ , we can *contract* T and T' along e to obtain an $(n + n' - 2)$ -tensor T'' , defined by

$$T''_{i_1, \dots, \widehat{i_k}, \dots, i_n, j_1, \dots, \widehat{j_{k'}}, \dots, j_{n'}} := \sum_{i_k = j_{k'} \in \{1, \dots, m_k\}} T_{i_1, \dots, i_k, \dots, i_n} T'_{j_1, \dots, j_{k'}, \dots, j_{n'}}$$

where the hats indicate that the corresponding indices are deleted. Moreover, we can remove the edge e and replace T and T' and their corresponding vertices with T'' and a new vertex, obtaining a tensor network with one fewer vertex and edge than Γ .

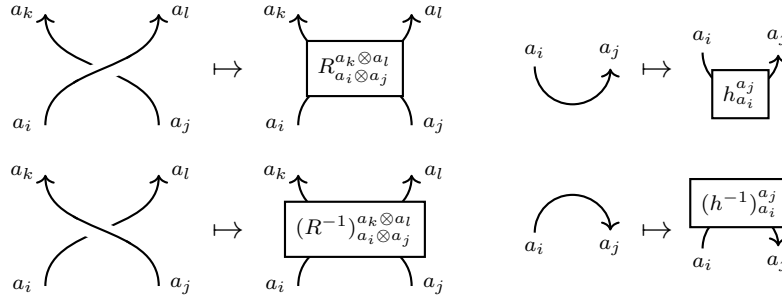
More generally, if legs $k_1, \dots, k_s$ of T and legs $k'_1, \dots, k'_s$ of T' are associated with the two ends of edges $e_1, \dots, e_s$ in Γ respectively, we can contract T and T' along $e_1, \dots, e_s$ at once

to obtain an $(n + n' - 2s)$ -tensor T'' defined by

$$T''_{i_1, \dots, \widehat{i_{k_1}}, \dots, \widehat{i_{k_s}}, \dots, i_n, j_1, \dots, \widehat{j_{k'_1}}, \dots, \widehat{j_{k'_s}}, \dots, j_{n'}} := \sum_{\substack{i_{k_1}=j_{k'_1} \in \{1, \dots, m_{k_1}\} \\ \vdots \\ i_{k_s}=j_{k'_s} \in \{1, \dots, m_{k_s}\}}} T_{i_1, \dots, i_n} T'_{j_1, \dots, j_{n'}} \quad (5)$$

We can then remove the edges $e_1, \dots, e_s$ and replace T and T' and their corresponding vertices with T'' and a new vertex, obtaining a tensor network with one fewer vertex and s fewer edges than Γ . Similarly, we can define contractions of more than two tensors at once.

Example 2.4. Given an R -matrix $R \in \text{Aut}(V \otimes V)$ and a ribbon element $h \in \text{Aut}(V)$ determining a Reshetikhin–Turaev functor, we fix a basis $\mathcal{B} := \{e_1, \dots, e_d\}$ of V . Under $\mathcal{B}$, $R^{\pm 1}$ becomes a 4-tensor $(R^{\pm 1})_{e_i \otimes e_j}^{e_k \otimes e_l}$ as (i, j, k, l) varies in $\{1, \dots, d\}^4$. Similarly $h^{\pm 1}$ becomes a 2-tensor $(h^{\pm 1})_{e_i}^{e_j}$. For an oriented long knot diagram with n crossings, the Reshetikhin–Turaev functor turns it into a tensor network by the following assignment:



That is, it turns each crossing into a degree 4 vertex associated with a 4-tensor $(R^{\pm 1})_{e_i \otimes e_j}^{e_k \otimes e_l}$, and each cup or cap going rightwards into a degree 2 vertex associated with a 2-tensor $(h^{\pm 1})_{e_i}^{e_j}$. With rotation numbers in $\{0, \pm 1\}$, we obtain a tensor network with no more than $2n$ vertices, where all legs are of dimension d .

Example 2.5. Figure 3 illustrates the tensor network obtained by applying the Reshetikhin–Turaev functor on an oriented long knot diagram of the 4_1 knot. With the symbols in Figure 3, one can check that the corresponding knot polynomial, as discussed at the end of Section 2.2, can be expressed as the following finite sum:

$$\sum_{\substack{a_1, \dots, a_9 \in \mathcal{B} \\ a_0 = a_{10} = e_1}} R_{a_0 \otimes a_7}^{a_8 \otimes a_1} \cdot h_{a_3}^{a_4} \cdot (R^{-1})_{a_4 \otimes a_8}^{a_9 \otimes a_5} \cdot R_{a_5 \otimes a_1}^{a_2 \otimes a_6} \cdot (h^{-1})_{a_6}^{a_7} \cdot (R^{-1})_{a_9 \otimes a_2}^{a_3 \otimes a_{10}}. \quad (6)$$

Recall that $\mathcal{B} = \{e_1, \dots, e_d\}$ is the basis of V .

Examples 2.4 and 2.5 show that all Reshetikhin–Turaev knot polynomials can be expressed as a sum similar to (6). The sum is finite, hence we can compute it piece by piece in an arbitrary order, which naturally translates into doing a sequence of tensor contractions on the tensor network until only one vertex remains. Such a sequence is called a *contraction sequence*. A contraction sequence can be encoded by an ordered list of sets of edges removed at each step, and it plays a crucial role in the time complexity of computing the contractions, which we discuss in detail in the following section. Note that we can always eliminate two

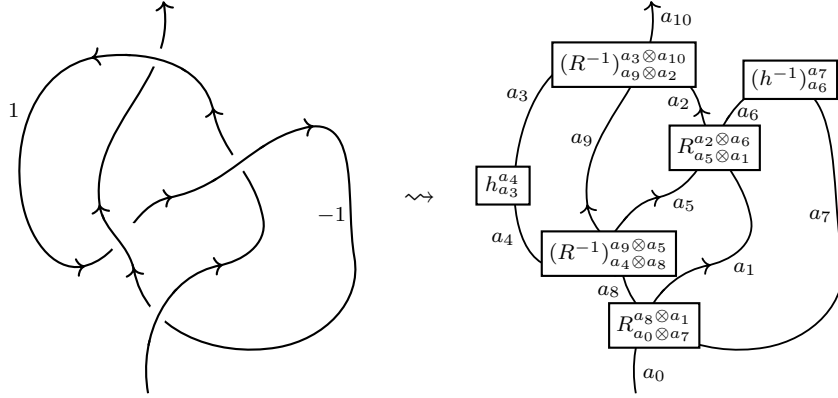


FIGURE 3. The tensor network resulted from an oriented long knot diagram of the 4_1 knot.

free legs of the tensor network by, for example, setting at the beginning $a_0 = a_{10} = e_1$ in Example 2.5.

3. BOUNDING THE TIME COMPLEXITY

3.1. Contraction-width. We now restrict ourselves to tensor networks obtained by applying the Reshetikhin–Turaev functor to oriented long knot diagrams, where all legs of tensors have dimension $d = \dim V$.

It is clear from (5) that it takes $O(c \cdot d^s)$ additions and multiplications in $\mathcal{R}$ to compute a single entry in a tensor resulting from contracting c tensors along s edges at once. Let $l_1, \dots, l_c$ be the number of legs of the c tensors, the resulting tensor thus has $-2s + \sum_{i=1}^c l_i$ legs, hence consists of $d^{-2s + \sum_{i=1}^c l_i}$ entries. Therefore the total number of additions and multiplications required to build the resulting tensor is

$$O(c \cdot d^s) \cdot d^{-2s + \sum_{i=1}^c l_i} = O(c \cdot d^{-s + \sum_{i=1}^c l_i}).$$

This motivates the following definition.

Definition 3.1. Given a contraction sequence $\mathcal{S}$ on a tensor network Γ , for a step in $\mathcal{S}$ that contracts c tensors with $l_1, \dots, l_c$ legs respectively along s edges at once, its *local contraction-width* is $-s + \sum_{i=1}^c l_i$. The *contraction-width* $w(\mathcal{S})$ of $\mathcal{S}$ is the maximum local contraction-width of all its steps, and the contraction-width $w(\Gamma)$ of Γ is the minimal contraction-width among all possible contraction sequences on Γ .

Note that the contraction-width is independent of the dimension d , hence it makes sense to consider (abstract) contraction sequences on a graph Γ and define its contraction-width $w(\Gamma)$ similarly. Note also that adding degree 2 vertices does not affect the contraction-width of a nontrivial graph. These justify the use of $w(D)$ in Section 1.

It is clear that adding more tensors to a step of contraction never decreases its local contraction-width, hence at best has no effect on the exponential factor of the number of additions and multiplications required. Therefore we further restrict ourselves to contraction sequences where each step contracts exactly 2 tensors. In particular, for a tensor network

with $2n$ vertices, all contraction sequences consist of $2n - 1$ steps. For a contraction sequence $\mathcal{S}$ with width w , it then takes $O(n \cdot d^w)$ additions and multiplications in $\mathcal{R}$ to perform all steps in $\mathcal{S}$.

In [MS08], Markov and Shi restricted themselves to contraction sequences where each step contracts only one edge. For such a contraction sequence $\mathcal{S}$, they defined its *contraction complexity* $cc(\mathcal{S})$ as the maximum degree of a merged vertex during the contraction process, and hence the contraction complexity $cc(\Gamma)$ of a tensor network Γ is the minimum complexity among all possible $\mathcal{S}$ where each step contracts along only one edge. It is immediate from definition that $w(\mathcal{S}) = cc(\mathcal{S}) + 1$, hence

$$w(\Gamma) \leq cc(\Gamma) + 1, \quad (7)$$

where the inequality may be strict since the contraction-width is defined over a larger family of contraction sequences. Recall that the *line graph* Γ^* of a graph Γ is defined by $V(\Gamma^*) := E(\Gamma)$ and

$$E(\Gamma^*) := \{\{e, f\} \subset E(\Gamma) \mid e \neq f, \exists v \in V(\Gamma) \text{ such that } v \in e, v \in f\}.$$

The following were proved in [MS08]:

Proposition 3.2 (Markov–Shi). For any graph Γ , $cc(\Gamma) = tw(\Gamma^*)$. Furthermore, given a tree decomposition of Γ^* of width w , there is a polynomial-time algorithm translating it into a contraction sequence $\mathcal{S}$ on Γ with $w(\mathcal{S}) \leq w$.

Lemma 3.3 (Markov–Shi). For any graph Γ of maximum degree $\Delta(\Gamma)$,

$$(tw(\Gamma) - 1)/2 \leq tw(\Gamma^*) \leq \Delta(\Gamma)(tw(\Gamma) + 1) - 1. \quad (8)$$

Therefore combining (7), Proposition 3.2 and Lemma 3.3, we have established that

$$w(\Gamma) \leq cc(\Gamma) + 1 = tw(\Gamma^*) + 1 \leq \Delta(\Gamma)(tw(\Gamma) + 1). \quad (9)$$

This gives Lemma 1.3 as we stated.

We end this section with a remark that the contraction width is also linearly bounded by the cut-width from above; more precisely, we have $w(\Gamma) \leq 3c(\Gamma)$, recalling that $c(\Gamma)$ denotes the cut-width of Γ . This is because that a diagram realizing the graph's cut-width gives a sequential order for the tensors, where the tensors are connected one by one and the legs of each tensor can be partitioned into two sides, where the number of legs on each side does not exceed $c(\Gamma)$. In particular, by fixing a small braid index, one can construct knots with arbitrarily many crossings while maintaining a small contraction-width.

3.2. Proof of Theorem 1.2. Given an oriented knot diagram D with n crossings and a contraction sequence on D with contraction-width w , we cut an arbitrary arc in D open, turning it into an oriented long knot diagram D' . As discussed in Section 2.2, the arc cut open can be eliminated immediately after we apply the Reshetikhin–Turaev functor to D' , obtaining a tensor network with at most $2n$ vertices along with a contraction sequence whose width is at most w . This procedure is $O(n)$ and is dominated by subsequent computation.

We have established in the previous section that it takes $O(n \cdot d^w)$ additions and multiplications in $\mathcal{R}$ to compute the desired knot polynomial, hence it remains only to bound the time complexity of performing additions and multiplications in $\mathcal{R}$.

For a k -variable Reshetikhin–Turaev knot polynomial, we have $\mathcal{R} = \mathbb{Z}[t_1^{\pm 1}, \dots, t_k^{\pm 1}]$. Let M be the maximum degree of all variables in the entries of $(R_{e_i \otimes e_j}^{e_k \otimes e_l})$ and $(h_{e_i}^{e_j})$. The maximum degree for each variable that can appear throughout the process is thus $2nM$. Hence all polynomials have at most $(2nM)^k = O(n^k)$ terms. Therefore the naive number of additions and multiplications in $\mathbb{Z}$ required for each polynomial addition or multiplication is $O(n^{2k})$. Alternatively, we use Kronecker substitution to turn multivariable polynomials into univariate ones and apply fast Fourier transform to compute their multiplications, which reduces the required number of additions and multiplications to $O(k(4nM + 1)^k \log(4nM + 1)) = O(n^k \log n)$ [vzGG99, Section 8.4].

We now bound the integers that may appear during the process. Let C be the maximum absolute value of coefficients of the entries of $(R_{e_i \otimes e_j}^{e_k \otimes e_l})$ and $(h_{e_i}^{e_j})$. Since there are at most M^k distinct monomials appearing in the entries, the maximum absolute value of integers that can appear throughout the process is bounded by the maximum absolute value of the coefficients of

$$d^{2n} \left(\sum_{i=1}^{M^k} C m_i \right)^{2n} = \sum_{\substack{p_1 + \dots + p_{M^k} = 2n \\ p_1, \dots, p_{M^k} \geq 0}} \frac{d^{2n} \cdot (2n)! \cdot C^{2n}}{(p_1!) \cdots (p_{M^k}!)} \cdot m_1^{p_1} \cdots m_{M^k}^{p_{M^k}}. \quad (10)$$

where each m_i represents a monic monomial. Hence it is bounded by

$$\sum_{\substack{p_1 + \dots + p_{M^k} = 2n \\ p_1, \dots, p_{M^k} \geq 0}} \frac{d^{2n} \cdot (2n)! \cdot C^{2n}}{(p_1!) \cdots (p_{M^k}!)} \leq \binom{2n + M^k}{M^k} \cdot (2ndC)^{2n} \leq (2ndCM^k)^{2n + M^k}, \quad (11)$$

so the bit-size of the integers is $O(n \log n)$. Therefore Theorem 1.2 follows from the fact that adding and multiplying integers with bit-size in $O(n \log n)$ are

$$O((n \log n) \cdot \log(n \log n)) = O(n(\log n)^2). \quad (12)$$

3.3. Proof of Corollary 1.4. We have established that, in order to efficiently compute Reshetikhin–Turaev knot polynomials with tensor networks, one needs to produce, on any given planar diagram D , a contraction sequence $\mathcal{S}$ whose width is as close to $w(D)$ as possible. We say that a contraction sequence $\mathcal{S}$ on D is *optimal* if $w(\mathcal{S}) = w(D)$. The task of finding an optimal contraction sequence on a given graph is known as the *tensor network contraction ordering*, and is NP-hard for general graphs [LSW97]; the author is not aware of any specific result for knot diagrams.

In practice, optimized algorithms of breadth-first or depth-first searches [PHV14], albeit scale exponentially, are able to produce optimal contraction sequences for knot diagrams up to 30 or more crossings. In theory, to obtain a statement such as Corollary 1.4, one can use approximation algorithms for contraction sequences instead of optimal ones, where an approximation algorithm for contraction sequences is an algorithm which, for any input graph Γ , gives a contraction sequence $\mathcal{S}$ such that $w(\mathcal{S}) \leq \alpha \cdot w(\Gamma)$ for some constant α . By Proposition 3.2, one can obtain approximation algorithms for contraction sequences from approximation algorithms for tree decompositions. A summary of different approximation algorithms for tree decompositions can be found in [Kor22], from which we use the following one here for Corollary 1.4:

Proposition 3.4 (Korhonen). There is an algorithm that, given an n -vertex graph Γ and an integer w , in time $2^{10.8w}n$ either outputs a tree decomposition of Γ of width at most $2w + 1$, or determines that the tree-width of Γ is larger than w .

Since constructing the line graph Γ^* is polynomial in the size of Γ , by constructing Γ^* , applying Proposition 3.4 to Γ^* , and then Proposition 3.2 to the tree decomposition returned, we obtain

Lemma 3.5. There is an algorithm that, given an n -vertex graph Γ and an integer w , in time $2^{10.8w}n$ either outputs a contraction sequence $\mathcal{S}$ of Γ with $w(\mathcal{S}) \leq 2w + 1$, or determines that the contraction width of Γ is larger than w .

Therefore Corollary 1.4 follows by combining Lemma 3.5 with Theorem 1.2.

Acknowledgements. The author wishes to thank Nathan Dunfield, Stavros Garoufalidis, Saul Schleimer, and Jonathan Spreer for useful conversations. The author also specially thanks Mingde Ren, from whom the author learned about tensor networks. This material is based upon work supported by the U.S. National Science Foundation under Grant No. DMS-2424139 while the author was in residence at the Simons Laufer Mathematical Sciences Institute in Berkeley, California, during the Spring 2026 semester. The author was also supported by the U.S. National Science Foundation under Grant No. DMS-2303572 during the Spring and Summer 2026 semesters.

REFERENCES

- [BC17] Jacob Bridgeman and Christopher Chubb, *Hand-waving and interpretive dance: an introductory course on tensor networks*, J. Phys. A **50** (2017), no. 22, 223001, 61.
- [Bie90] Dan Bienstock, *On embedding graphs in trees*, J. Combin. Theory Ser. B **49** (1990), no. 1, 103–136.
- [BN05] Dror Bar-Natan, *Khovanov’s homology for tangles and cobordisms*, Geom. Topol. **9** (2005), 1443–1499.
- [BNvdV24] Dror Bar-Natan and Roland van der Veen, *A perturbed-Alexander invariant*, Quantum Topol. **15** (2024), no. 3, 449–472.
- [CDGW] Marc Culler, Nathan Dunfield, Matthias Goerner, and Jeffrey Weeks, *SnapPy, a computer program for studying the geometry and topology of 3-manifolds*, Available at <http://snappy.computop.org>.
- [CFK⁺15] Marek Cygan, Fedor V. Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh, *Parameterized algorithms*, Springer, Cham, 2015.
- [GK26] Stavros Garoufalidis and Rinat Kashaev, *Multivariable Knot Polynomials from Braided Hopf Algebras with Automorphisms*, Publ. Res. Inst. Math. Sci. **62** (2026), no. 1, 75–114.
- [GL24] Stavros Garoufalidis and Shana Li, *Values of V_n -polynomials of knots*, <https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/XE4TOF>, 2024.
- [GL26] Stavros Garoufalidis and Shana Yunsheng Li, *Patterns of the V_2 -polynomial of knots*, Experimental Mathematics (2026), 2409.03557, to appear.
- [GLY] Stavros Garoufalidis, Shana Yunsheng Li, and Josephine Yu, *Positivity and concavity of the colored Links–Gould polynomials of knots*, In preparation.
- [HTW98] Jim Hoste, Morwen Thistlethwaite, and Jeff Weeks, *The first 1,701,936 knots*, Math. Intelligencer **20** (1998), no. 4, 33–48.
- [Jon87] Vaughan Jones, *Hecke algebra representations of braid groups and link polynomials*, Ann. of Math. (2) **126** (1987), no. 2, 335–388.

- [JWV90] F. Jaeger, D. L. Vertigan, and D. J. A. Welsh, *On the computational complexity of the Jones and Tutte polynomials*, Math. Proc. Cambridge Philos. Soc. **108** (1990), no. 1, 35–53.
- [Kor22] Tuukka Korhonen, *A single-exponential time 2-approximation algorithm for treewidth*, 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science—FOCS 2021, IEEE Computer Soc., Los Alamitos, CA, [2022] ©2022, pp. 184–192.
- [KR89] A. N. Kirillov and N. Yu. Reshetikhin, *Representations of the algebra $U_q(\mathfrak{sl}(2))$, q -orthogonal polynomials and invariants of links*, Infinite-dimensional Lie algebras and groups (Luminy-Marseille, 1988), Adv. Ser. Math. Phys., vol. 7, World Sci. Publ., Teaneck, NJ, 1989, pp. 285–339.
- [KS93] Ephraim Korach and Nir Solel, *Tree-width, path-width, and cutwidth*, Discrete Appl. Math. **43** (1993), no. 1, 97–101.
- [Li26] Shana Yunsheng Li, *Spherogram*, <https://github.com/Shakugannotorch/Spherogram>, 2026.
- [LSW97] Chi-Chung Lam, P. Sadayappan, and Rephael Wenger, *On optimizing a class of multi-dimensional loops with reduction for parallel execution*, Parallel Process. Lett. **7** (1997), no. 2, 157–168.
- [LT79] Richard J. Lipton and Robert Endre Tarjan, *A separator theorem for planar graphs*, SIAM J. Appl. Math. **36** (1979), no. 2, 177–189.
- [Mak05] J.A. Makowsky, *Coloured tutte polynomials and kauffman brackets for graphs of bounded tree width*, Discrete Applied Mathematics **145** (2005), no. 2, 276–290, Structural Decompositions, Width Parameters, and Graph Labelings.
- [Mar21] Clément Maria, *Parameterized complexity of quantum knot invariants*, 37th International Symposium on Computational Geometry, LIPIcs. Leibniz Int. Proc. Inform., vol. 189, Schloss Dagstuhl. Leibniz-Zent. Inform., Wadern, 2021, pp. Art. No. 53, 17.
- [MK19] Konstantinos Meichanetzidis and Stefanos Kourtis, *Evaluating the Jones polynomial with tensor networks*, Phys. Rev. E **100** (2019), no. 3, 033303, 7.
- [MS08] Igor L. Markov and Yaoyun Shi, *Simulating quantum computation by contracting tensor networks*, SIAM J. Comput. **38** (2008), no. 3, 963–981.
- [Oht02] Tomotada Ohtsuki, *Quantum invariants*, Series on Knots and Everything, vol. 29, World Scientific Publishing Co., Inc., River Edge, NJ, 2002, A study of knots, 3-manifolds, and their sets.
- [Orú19] Román Orús, *Tensor networks for complex quantum systems*, Nature Reviews Physics **1** (2019), no. 9, 538–550.
- [PHV14] Robert Pfeifer, Jutho Haegeman, and Frank Verstraete, *Faster identification of optimal contraction sequences for tensor networks*, Physical Review E **90** (2014), no. 3, 033315– (eng).
- [RT90] Nikolai Reshetikhin and Vladimir Turaev, *Ribbon graphs and their invariants derived from quantum groups*, Comm. Math. Phys. **127** (1990), no. 1, 1–26.
- [Shu11] Alexander N. Shumakovitch, *Patterns in odd Khovanov homology*, J. Knot Theory Ramifications **20** (2011), no. 1, 203–222.
- [Tur88a] V. G. Turaev, *The Yang-Baxter equation and invariants of links*, Invent. Math. **92** (1988), no. 3, 527–553.
- [Tur88b] ———, *The Yang-Baxter equation and invariants of links*, Invent. Math. **92** (1988), no. 3, 527–553.
- [vzGG99] Joachim von zur Gathen and Jürgen Gerhard, *Modern computer algebra*, Cambridge University Press, New York, 1999.
- [Wit89] Edward Witten, *Quantum field theory and the Jones polynomial*, Braid group, knot theory and statistical mechanics, Adv. Ser. Math. Phys., vol. 9, World Sci. Publ., Teaneck, NJ, 1989, pp. 239–329.

(Shana Yunsheng Li) DEPARTMENT OF MATHEMATICS, UNIVERSITY OF ILLINOIS, URBANA, IL, USA
<https://shana-y-li.github.io>

Email address: yl202@illinois.edu